

Vector Embeddings & ChromaDB Study Guide

Comprehensive Architectural & Technical Reference | Ziad Rahbani RAG System

1. Core Foundations: Vectors & Text Embeddings

Vector Definition & Mathematical Representation

In computer science and linear algebra, a **Vector** is an ordered list of numerical values representing a coordinate point in an n -dimensional space. When applied to Natural Language Processing (NLP), a vector acts as a dense numerical fingerprint of a piece of text.

Embeddings: Lexical vs. Semantic Representation

- **Lexical Representation (Traditional Search):** Methods like TF-IDF or BM25 rely on exact keyword matching. They fail when queries use synonyms, slang, or alternative phrasing (e.g., searching for "automobile" will miss documents containing only "car").
- **Dense Semantic Embeddings:** Transforming text into continuous vector representations maps concepts based on context and meaning. Words or chunks with similar semantic intents are positioned close to one another within the vector space, regardless of exact keyword overlap.

The 4-Step Text-to-Vector Transformation Pipeline

- 1. Subword Tokenization:** The raw string is segmented into smaller units called *tokens* using algorithms such as Byte-Pair Encoding (BPE) or WordPiece. Each token is assigned a unique numerical ID from the model's vocabulary dictionary.
- 2. Static Embedding Lookup:** Each Token ID fetches an initial static vector from the model's embedded lookup matrix. At this stage, word context is not yet incorporated.
- 3. Contextual Encoding via Self-Attention:** Token vectors pass through multiple Transformer encoder layers. Multi-Head Self-Attention calculates Query (**Q**), Key (**K**), and Value (**V**) matrices, allowing each token to dynamically absorb contextual meaning from surrounding tokens in the sentence.
- 4. Mean Pooling (Vector Aggregation):** After contextual processing, the model outputs individual vectors for every token in the sequence. **Mean Pooling** calculates the mathematical average across all token vectors (excluding padding tokens) to produce a single, fixed-length vector representing the entire chunk.

2. Vector Similarity & Distance Metrics

To retrieve relevant chunks during a search, vector databases calculate the distance between a query vector and stored document vectors using mathematical metrics:

Metric	Mathematical Concept	Key Characteristic & Ideal Use Case
Cosine Similarity	$Cosine(\theta) = (A \cdot B) / (\ A\ \ B\)$	Scale-invariant; evaluates direction and semantic alignment rather than magnitude (range: -1 to 1). Ideal for normalized text embeddings.
Euclidean Distance (L2)	$d(A, B) = \sqrt{\sum (A_i - B_i)^2}$	Measures straight-line distance in n-d space. Sensitive to vector length and magnitude. Best for spatial clustering.
Dot Product	$A \cdot B = \sum (A_i \times B_i)$	Combines angle and magnitude; equivalent to Cosine Similarity when vectors are unit-normalized. Extremely fast computation.

3. Model Architecture & Ecosystem Analysis

The Hugging Face Framework

Hugging Face acts as an open-source hub for machine learning models, tokenizers, and datasets. It provides standardized abstractions to download, load, and run pretrained neural networks locally without external API dependencies.

Technical Analysis: sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2

- **Sentence-Transformers Framework:** A specialized modification of BERT-based networks using bi-encoder architectures to derive semantically meaningful sentence embeddings that can be compared using Cosine Similarity.
- **Paraphrase Training Objective:** Pretrained on massive datasets of paraphrased sentence pairs, enabling the model to group distinct sentences that express the same underlying idea together.
- **Multilingual Space Alignment:** Trained using teacher-student multilingual knowledge distillation. The model aligns over 50 languages (including Arabic and English) into a shared vector space, allowing cross-lingual retrieval and robust handling of regional dialects.
- **MiniLM-L12 Architecture:** A lightweight Transformer model featuring 12 encoder layers. Produced via Knowledge Distillation—compressing a large "teacher" model into a smaller "student" model while retaining over 95% of its semantic performance.
- **384-Dimensional Space:** Each input text is compressed into a vector of exactly 384 floating-point numbers. This dimension count offers an optimal trade-off between semantic expressive power and low computational/memory overhead for local CPU execution.

4. Vector Database Architecture: ChromaDB

Relational Databases vs. Vector Databases

- **Relational Databases (e.g., SQLite, PostgreSQL):** Optimized for structured data point lookups and exact filter criteria using B-Tree indexing ($O(\log N)$ for exact matches).
- **Vector Databases (e.g., ChromaDB):** Optimized for high-dimensional spatial indexing and Approximate Nearest Neighbor (ANN) searches across millions of floating-point arrays.

ChromaDB On-Disk Folder Structure (`ziad_chroma_db`)

- `chroma.sqlite3` : A relational SQLite file managing system operational states, collection details, original chunk text payloads (`documents`), user-defined metadata dictionaries (`metadatas`), and unique identifier strings (`ids`).
- **Binary Index Files (`.bin` files):** Raw binary structures storing floating-point vector arrays and spatial index nodes managed by the vector engine (e.g., `data_level0.bin` , `link_lists.bin`).

Vector Indexing Engine: HNSW Algorithm

ChromaDB uses the **Hierarchical Navigable Small World (HNSW)** graph algorithm to index high-dimensional vectors efficiently:

1. **Multi-Layer Graph Structure:** HNSW constructs a multi-layered graph similar to a Skip List data structure. The top layers contain long-range "highway" connections between distant vectors, while the bottom layers contain dense connections between close neighbors.
2. **Search Traversal Mechanism:** Search begins at the sparse top layer to traverse large vector distances quickly. Once the search approaches the target region, it drops down layer by layer to perform fine-grained neighbor matching.
3. **Logarithmic Efficiency:** HNSW avoids brute-force linear scans ($O(N)$ comparisons) by evaluating queries in $O(\log N)$ logarithmic time complexity, enabling sub-second retrieval across massive datasets.

5. The Retrieval Lifecycle in a RAG System

1. **Query Ingestion:** The user submits a natural language question.
2. **Query Vectorization:** The query text passes through the same `paraphrase-multilingual-MiniLM-L12-v2` embedding model instance, projecting the question into the identical 384-dimensional vector space.
3. **ANN Graph Traversal:** ChromaDB uses the HNSW graph index to locate the K closest document vectors based on distance metrics.
4. **Payload Resolution:** ChromaDB maps the winning vector IDs back to `chroma.sqlite3`, retrieving the original text chunks and associated source metadata.
5. **Context Injection:** The retrieved text chunks are injected into the Large Language Model (LLM) system prompt to generate an accurate, grounded response.